

PRD: ProdAlert AI – Autonomous SRE Agent

2026-09-17 · @Someone

1. Overview and problem

ProdAlert AI is an AI agent that investigates P1 production alerts within 60 seconds, so engineers start with a root cause instead of raw logs.

Today's problem, step by step:

1. A P1 alert fires in production (for example: CPU high, database down, payment failures).
2. The on-call engineer gets a phone call, often at 2 AM.
3. They spend about 30 minutes reading logs, searching old tickets and runbooks, and guessing the cause.
4. Only then do they apply a fix.

Result: Mean Time To Resolve (MTTR) is around 30 minutes per P1. Engineers are tired, and the same incidents are investigated again and again.

What we will build: When an alert arrives, the agent reads the logs, finds similar past incidents, explains the likely root cause, creates a Jira ticket, posts to Slack, and (only for approved safe actions) runs a fix such as restarting a pod.

2. Goals, non-goals and success metrics

The main goal is to cut MTTR for P1 alerts from about 30 minutes to under 2 minutes, without the agent ever making things worse.

Goals

1. Start analysis within 5 seconds of the alert and finish within 60 seconds.
2. Give a clear root cause with evidence (log lines and similar past incidents).
3. Create a Jira ticket and a Slack message automatically.
4. Run only pre-approved, low-risk fixes (for example, restart a pod) automatically.
5. Record every step so humans can review what the agent did and why.

Non-goals (version 1)

- Handling P2/P3 alerts.
- Writing or deploying new code fixes.

- Risky actions such as database failover, rollbacks or deleting data without a human approving.
- Replacing the on-call engineer. The engineer still owns the incident.

Success metrics

Metric	Target	How we measure
MTTR for P1	< 2 min	Alert time to resolved time in PagerDuty
Time to first diagnosis	< 60 sec	Job created to Slack message sent
Root-cause accuracy	> 80% (stretch 84%)	Mandatory engineer review form per incident (section 8.1)
Hallucination rate	< 5% in eval, 0 critical in prod	DeepEval faithfulness + human review
Tokens per incident	< 20k	LangSmith trace
Wrong auto-remediations	0	Remediation audit log

3. Users and user flow

The primary user is the on-call SRE; secondary users are service owners and engineering managers who read the Jira and Slack output.

```

flowchart LR
    A[Prometheus / PagerDuty<br/>alert] --> B[FastAPI<br/>/api/v1/alert]
    B --> C[Redis + Celery<br/>agent_queue]
    C --> D[LangGraph agent]
    D --> E[Jira ticket]
    D --> F[Slack message]
    D --> G[Safe remediation]
    D --> H[LangSmith trace]
```

The alert flows left to right; the agent is the only component that makes decisions.

Step by step

1. **Alert fires.** Prometheus or PagerDuty detects a P1 problem and calls our webhook.
2. **API accepts it.** FastAPI checks the payload, saves a job row in Postgres, and replies at once with a `job_id` and status `analyzing`.
3. **Job is queued.** The job is pushed to `agent_queue` in Redis so the API stays fast.

4. **Worker picks it up.** A Celery worker takes the job and starts the LangGraph agent.
5. **Logs are read.** The agent queries Loki (plus Prometheus metrics for resource alerts) for the alert's time window and uses Haiku to shrink the logs to the important errors.
6. **Past incidents are searched.** The agent searches Pinecone for similar incidents, runbooks and Jira tickets.
7. **Root cause is decided.** Sonnet reads the log summary and past fixes and returns a root cause, confidence score and suggested fix.
8. **Action is taken.** The agent creates a Jira ticket, posts to Slack, and runs a remediation only if it is on the safe list and confidence is high.
9. **Engineer reviews.** The on-call engineer reads the Slack message, confirms or corrects, and closes the incident.
10. **System learns.** The final resolution is embedded into Pinecone through `ingestion_queue` for future incidents.

4. Architecture

The system has five layers: ingestion, queue, agent, memory and actions. Each can be scaled and tested on its own.

A. Ingestion layer (FastAPI)

- Endpoint: `POST /api/v1/alert`
- Request body: `{"service": "payments-api", "alert_type": "memory_high", "severity": "P1", "fired_at": "2026-09-17T02:03:00Z", "loki_query": "{service=\"payments-api\"} |= \"error\"}"`
- Response (under 200 ms): `{"job_id": "123", "status": "analyzing"}` (or `{"job_id": "123", "status": "deduplicated"}` for a repeat alert)
- Also: `GET /api/v1/jobs/{job_id}` to check status and result.
- Checks: webhook signature, payload schema (Pydantic), P1 only, then the dedupe and rate-limit rules below.

A1. Log source (Phase 1: Loki only)

Phase 1 supports exactly one log source: Grafana Loki. ELK and CloudWatch are out of scope until Phase 2.

- **Why Loki:** it pairs with Prometheus (same labels such as `service`, `namespace`, `pod`), and LogQL can be built from the alert labels without a human.
- **How logs are fetched:** `log_reader` calls the Loki `query_range` API with the alert's labels and a time window of `fired_at - 15 min` to `fired_at + 2 min`.

- **Limits:** max 5,000 lines per query; if more, fetch only `level=error|warn` lines plus the last 500 lines of the pod.
- **Real-time case (example: RAM at 99%):** logs alone rarely show the cause of memory pressure. For `memory_high` and `cpu_high`, `log_reader` also pulls from Prometheus: container memory over the last 30 minutes, pod restarts, OOMKilled events and the most recent deploy time. The agent diagnoses from logs + metrics together.
- **If Loki is down or returns nothing:** the agent posts "logs unavailable" in Slack, gives a metrics-only hint with low confidence, and never auto-remediates.
- **Phase 2:** a `LogSource` interface with Loki, Elasticsearch and CloudWatch adapters, chosen per service in config.

A2. Dedupe and rate limiting

One incident = one agent run: if the same service sends a RAM 99% alert 10 times in 2 minutes, exactly 1 agent runs and the other 9 alerts attach to that job.

Layer 1: Fingerprint dedupe (same alert repeating)

1. Build a fingerprint: `sha256(service + alert_type + namespace)`.
2. Try to take a Redis lock: `SET dedupe:<fingerprint> <job_id> NX EX 900`. `NX` makes this atomic, so two alerts arriving at the same millisecond cannot both win.
3. **Lock taken:** create the job and queue it.
4. **Lock already exists:** do not queue. Increment `alert_count` on the existing job, append the new `fired_at`, and return `status: deduplicated` with the existing `job_id`.
5. The lock lasts 15 minutes and is released early when the job's incident is resolved. A new alert after that is a new incident.
6. If the repeat count reaches 10 within the window, post one Slack update ("alert still firing, 10 times"), not 10 messages.

Layer 2: Rate limits (different alerts, same problem)

A RAM problem often triggers several alert types at once (`memory_high`, `pod_restart`, `latency_high`). Fingerprints differ, so Layer 1 alone would start 3 agents.

Limit	Value	When hit
Active agent jobs per service	1	New alert is attached to the running job as extra context
Agent runs per service per hour	5	Alert is recorded, no agent run; Slack says "rate limited, handle manually"

Limit	Value	When hit
Agent runs across all services per 10 minutes	20	Treated as a wide outage: agent pauses, one summary message to the incident channel, humans take over
Token budget per hour (all jobs)	500k tokens	New runs are skipped until the hour resets

- Counters use Redis sliding windows (`INCR` + `EXPIRE` , keyed by service and time bucket).
- Every deduplicated or rate-limited alert is still saved in Postgres, so nothing is lost.
- The normal PagerDuty page is never rate-limited; limits apply only to the agent.

B. Queue layer (Redis + Celery)

Queue	Purpose	Priority
<code>agent_queue</code>	Run the agent for a new alert	High
<code>ingestion_queue</code>	Embed new logs, runbooks and closed tickets into Pinecone	Low

- Workers for each queue run separately, so heavy embedding never slows down live incidents.
- Retries: 3 tries with backoff for tool failures; after that, the job goes to a dead-letter state and a human is paged.
- Timeout: a job is cancelled after 120 seconds and falls back to a normal page.

C. Agent layer (LangGraph)

```

flowchart LR
    S[Start] --> L[LogAnalyzer]
    L --> D[Diagnosis]
    D -->|confidence high| A[Action]
    D -->|confidence low| H[Escalate to human]
    A --> E[End]
    H --> E

```

A `StateGraph` passes one shared state object (alert, log summary, similar incidents, diagnosis, actions taken) between nodes.

1. **LogAnalyzer node.** Tools: `log_reader` , `vector_search` . Haiku parses raw logs into a short error summary. Pinecone returns the top similar incidents (we retrieve up to 100, then re-rank and keep the best 5 to stay within the token budget).
2. **Diagnosis node.** Model: Claude Sonnet. Prompt idea: "Given these logs and past fixes, what is the root cause? Cite the evidence. Stay under 20k tokens." Output is structured JSON: `root_cause` , `evidence` , `confidence` (0 to 1), `suggested_action` .
3. **Action node.** Tools: `create_jira_tool` , `send_slack_tool` , `run_remediation_script_tool` . Jira and Slack always run. Remediation runs only when the guardrails in section 5 pass.

D. Memory

Type	Store	What it holds	Lifetime
Short-term	Redis	Current incident context and state	Until incident closes (TTL 24h)
Long-term	Pinecone	Runbooks and past Jira tickets, embedded	Rolling 6 months
Job records	Postgres	Job status, diagnosis, actions, timings	Permanent
Agent instructions	<code>CLAUDE.md</code> / system prompt file	Rules, tone, safe action list	Versioned in git

E. Observability

- LangSmith: full trace of every agent run (inputs, tool calls, tokens, cost, latency).
- Structlog: JSON logs with `job_id` on every line so one incident can be followed end to end.

5. Safety guardrails for auto-remediation

The agent may only run a fix when every one of these checks passes; otherwise it recommends the fix in Slack and waits for a human.

1. **Allow-list only.** The action is on a fixed list (for example: restart one pod, scale a deployment by +1, clear a cache). The LLM picks from the list; it never writes shell commands.
2. **Confidence gate.** Diagnosis confidence is 0.85 or higher.
3. **Blast radius limit.** One service, one namespace, max 1 action per incident.

4. **Rate limit.** No more than 3 automatic actions for the same service per hour.
5. **Kill switch.** An environment flag `AUTO_REMEDIATION_ENABLED=false` stops all automatic actions instantly.
6. **Verify after acting.** The agent checks the alert metric 2 minutes later. If it has not improved, it pages the on-call engineer.
7. **Least privilege.** The remediation service account can only perform allow-listed actions.
8. **Audit.** Every action is logged with who (agent), what, why, and the trace link.

In shadow mode, remediation is fully disabled: the agent only posts what it *would* have done.

6. Tech stack and cost optimization

The stack is Python-first and runs locally with one `make up` command.

Area	Choice
Language	Python 3.11, managed with <code>uv</code>
Agent	LangGraph, LangChain, Claude API
API	FastAPI
Queue	Celery, Redis
Databases	Pinecone (vectors), Postgres (jobs)
DevOps	Docker, docker-compose, Makefile, <code>.env</code>
Code quality	Ruff, mypy, SonarQube, pytest, pytest-cov (80% coverage)
Observability	LangSmith, Structlog

How we keep token cost under 20k per incident

1. **Right model for each job.** Haiku parses and summarizes logs (cheap, fast). Sonnet does only the diagnosis.
2. **Prompt caching.** The system prompt, agent rules and tool definitions are cached, so repeated calls do not pay for them again.
3. **Tool filtering.** Each node gets only the tools it needs (LogAnalyzer never sees the remediation tool).

4. **Trim the context.** Send log summaries, not raw logs; send the top 5 similar incidents, not 100.
5. **Fresh context per incident.** Each job starts with a clean state, so no old incident leaks into a new one. (This is the runtime version of `/clear`, which is a Claude Code command and does not exist in the API.)

7. Testing and evaluation plan

We test in three layers, and the agent does not go to production until the eval targets pass twice in a row.

Layer	What we test	Tools	Pass criteria
Unit	Each tool (Jira, Slack, remediation) and each node alone	pytest, pytest-mock	All pass, coverage $\geq$ 80%
Integration	Full flow: fake alert -> API -> queue -> agent -> mocked Jira/Slack	docker-compose test stack	Job completes < 60 sec, correct records in Postgres
Eval	Diagnosis quality on real past incidents	Golden dataset, DeepEval, LangSmith	Targets below

Golden dataset (150 past incidents, minimum 100)

Pinecone holds about 2,000 tickets from 6 months, so 50 test cases would be too few to trust: with 50 cases, an 84% score could really be anywhere from about 71% to 92%. With 150 cases, the range narrows to about 77% to 89%.

1. **Size:** 150 resolved P1/P2 incidents (minimum 100 to start eval), about 7% of the 2,000-ticket history.
2. **Stratified by alert type**, so one common type cannot hide failures on rare ones. Target mix: ~30 memory/CPU, ~30 database, ~25 payment/external API, ~25 deploy-related, ~20 network/latency, ~20 other. Accuracy is reported per type, and every type must be at least 70%.
3. **Only human-confirmed labels:** each case has the root cause written in the closed Jira ticket and confirmed by a second engineer.
4. **Saved per case:** alert payload, Loki log snapshot, Prometheus metric snapshot, correct root cause, correct fix, and whether auto-remediation would have been safe.

5. **No leakage:** use a time split. Pinecone for eval is built only from tickets older than the test incidents, and the test incidents themselves plus their duplicates are removed.
6. **Include hard cases:** at least 20 cases where the right answer is "not enough evidence, escalate", to test that the agent does not guess.
7. **Grow it:** every thumbs-down or "wrong" review from shadow mode and production is added, targeting 300 cases by end of Phase 4.
8. **Report with a confidence interval**, for example "84% (95% CI 77-89%, n=150)", never a bare number.

Eval metrics

Metric	Target
Root-cause accuracy vs human	>= 84%
Faithfulness (DeepEval)	Claims supported by logs or retrieved incidents
Hallucination rate	< 5%
Tokens per incident	< 20k
Correct remediation choice	100% of auto-actions are on the allow-list and correct

CI rules

- Every pull request runs Ruff, mypy, unit tests and coverage.
- Changes to prompts or models also run the eval set; a drop of more than 3 accuracy points blocks the merge.

8. Rollout plan

We roll out in four phases, and each phase must meet its exit criteria before the next one starts.

Phase	What happens	Auto-remediation	Exit criteria
1. Build	Ingestion, queue, agent, tools, tests	Off	Unit + integration tests green, coverage >= 80%

Phase	What happens	Auto-remediation	Exit criteria
2. Offline eval	Run on the 150-incident golden dataset (min 100)	Off	Accuracy $\geq 84\%$ overall and $\geq 70\%$ per alert type, hallucination $< 5\%$, $< 20k$ tokens
3. Shadow mode (2 weeks)	Agent runs on real alerts, posts to a private Slack channel only; humans still handle incidents	Off	100% review coverage, ≥ 40 reviewed incidents, accuracy $> 80\%$ from mandatory reviews, 0 unsafe suggestions (see 8.1)
4. Assisted production	Agent posts to the main incident channel and creates Jira tickets	On for 1-2 safest actions only	0 wrong auto-actions for 30 days, MTTR trending to < 2 min

8.1 How shadow-mode accuracy is measured

Accuracy in shadow mode comes only from mandatory human reviews, one per incident; thumbs-up/down reactions are not used as the metric.

Step 1: Engineer works blind. The shadow diagnosis goes to a private channel and stays hidden from the on-call engineer until they record their own root cause. This stops the agent from biasing the human answer.

Step 2: Mandatory review form. When the engineer resolves the incident, Slack opens a short form (a Slack modal) linked to the `job_id`:

Field	Options
Agent root cause was	Correct / Partially correct / Wrong / Agent said "not enough evidence"
Actual root cause	Free text (required if not Correct)
Suggested fix would have	Worked / Partly worked / Not worked / Been unsafe
Any made-up facts (hallucination)?	Yes (describe) / No

Step 3: Enforced, not optional. The Jira ticket cannot move to Done until the review is submitted (Jira workflow validator checks a `prodalert_review_id` field). Missing reviews get a reminder after 4 hours and are escalated to the team lead after 24 hours.

Step 4: Second check on disagreements. Any "Wrong" or "Partially correct" review, plus a random 10% of "Correct" reviews, is checked by a second engineer in the weekly review. If they disagree, the second reviewer's label is final.

Step 5: Calculate the metrics automatically (daily job from Postgres `reviews` table to a dashboard):

- **Accuracy** = $\text{Correct} \div \text{total reviewed incidents}$. Partially correct counts as wrong for the headline number and is reported separately.
- **Coverage** = $\text{reviewed incidents} \div \text{total P1 incidents}$. Must be 100%; a missing review counts as Wrong.
- **Hallucination rate** = $\text{reviews marked "made-up facts: Yes"} \div \text{total reviewed}$.
- **Unsafe suggestion count** = fixes marked "Been unsafe". Must be 0.
- Also run DeepEval faithfulness automatically on every shadow run as an early warning, but it never replaces the human label.

Step 6: Minimum sample. Two weeks may not produce enough P1s. Shadow mode ends only when both conditions hold: at least 2 weeks and at least 40 reviewed incidents (P1 and P2 both count). If there are fewer, it is extended.

After phase 4, we add more allow-listed actions one at a time, each with its own review.

9. Runbook

This runbook tells the on-call engineer how to run ProdAlert AI, how to act on its output, and what to do when the agent itself fails.

9.1 Local setup

1. Clone the repo and install tools: `uv sync`.
2. Copy the env file: `cp .env.example .env`, then fill in `ANTHROPIC_API_KEY`, `PINECONE_API_KEY`, `JIRA_TOKEN`, `SLACK_BOT_TOKEN`, `LOKI_URL`, `PROMETHEUS_URL`, `LANGSMITH_API_KEY`, `DATABASE_URL`, `REDIS_URL`.
3. Start everything: `make up` (runs docker-compose with API, Redis, Postgres, workers).
4. Run checks: `make lint` (Ruff + mypy) and `make test` (pytest + coverage).
5. Send a fake alert: `make fake-alert SERVICE=payments-api TYPE=db_down`.
6. Check the result: `curl localhost:8000/api/v1/jobs/<job_id>` and open the LangSmith trace.

9.2 When a P1 alert fires (on-call steps)

1. **Acknowledge the page** in PagerDuty as usual. The agent does not replace this step.

2. **Open the Slack message** from ProdAlert AI (arrives within about 60 seconds). It shows root cause, confidence, evidence, Jira link and any action taken.
3. **Check the confidence.**
 - 0.85 or higher and an action was taken: watch the metric for 2 minutes to confirm recovery.
 - Below 0.85: treat the diagnosis as a hint. Follow the suggested fix manually after checking the evidence.
4. **Verify the evidence.** Open the log lines and similar incidents linked in the message. Make sure they match what you see.
5. **Fix or escalate.** If the diagnosis is wrong, fix the incident the normal way and escalate to the service owner if needed.
6. **Give feedback.** Submit the mandatory review form in Slack (Correct / Partially correct / Wrong + actual root cause). Jira cannot be closed without it.
7. **Close the incident.** Resolve in PagerDuty and Jira. The closed ticket is embedded into Pinecone automatically.

9.3 Service-specific playbooks

Alert	Agent usually checks	Safe auto-action	Needs a human
CPU high	Traffic spike, stuck process, recent deploy	Restart pod, scale +1 replica	Rollback of a bad deploy
DB down	Connection pool errors, disk full, primary health	None	Failover, disk expansion, restore
Payment failures	Gateway errors, timeouts, expired credentials	Restart payment worker pod	Gateway/provider contact, config change

9.4 When ProdAlert AI itself has problems

Symptom	Likely cause	What to do
No Slack message after 2 min	Worker down or queue stuck	Handle the incident manually. Then check <code>make status</code> , Celery worker logs, and Redis queue length.
Job stuck in <code>analyzing</code>	Claude or Pinecone API slow or down	Job times out at 120 sec. Check LangSmith trace for the failed step; check provider status pages.

Symptom	Likely cause	What to do
Wrong or strange diagnosis	Bad retrieval or hallucination	Ignore it, fix manually, mark thumbs-down, and add the case to the golden dataset.
Agent took a wrong action	Guardrail gap	Set <code>AUTO_REMEDIATION_ENABLED=false</code> immediately, undo the action, open a P1 bug on the agent.
Token cost above 20k	Too much log text or too many retrieved docs	Check the trace; lower top-k or tighten the Haiku summary step.
Duplicate tickets	Dedupe key not working	Close duplicates; check the Redis <code>'dedupe:'</code> lock (must use <code>'NX'</code>) and the per-service active-job limit.

9.5 Emergency: turn the agent off

1. Set `AUTO_REMEDIATION_ENABLED=false` and redeploy (or flip the feature flag) to stop all actions.
2. If needed, set `AGENT_ENABLED=false` so alerts go straight to normal paging.
3. Post in the incident channel that the agent is disabled and why.
4. Open a Jira ticket for the agent team with the job ID and LangSmith trace link.

9.6 Regular maintenance

- **Daily:** check the dead-letter jobs and failed traces in LangSmith.
- **Weekly:** review all thumbs-down cases; add good examples to the golden dataset.
- **Monthly:** re-run the full eval; remove Pinecone data older than 6 months; review the safe action allow-list.
- **On every prompt or model change:** run the eval set before merging.

10. Risks, open questions and resume point

The biggest risk is a confident but wrong diagnosis leading to a wrong action, which the guardrails and shadow mode are designed to catch.

Risks and mitigations

Risk	Mitigation
Hallucinated root cause	Evidence must be cited; faithfulness eval; human confirms
Wrong auto-action	Allow-list, confidence gate, kill switch, post-action check
Secrets or PII in logs sent to the LLM	Redact tokens, emails and card numbers before sending
Claude or Pinecone outage	120 sec timeout, normal paging continues
Stale runbooks in Pinecone	Monthly refresh; closed tickets embedded automatically
Engineers trust the agent too much	Engineer still acknowledges every P1; feedback loop
Engineers skip reviews, so accuracy cannot be trusted	Review form required to close Jira; missing review counts as Wrong
Alert storm starts many agent runs	Redis NX dedupe lock, 1 active job per service, global rate limits

Open questions

- Which 1-2 remediation actions go on the allow-list first?
- Who owns the golden dataset and the weekly second-review of disagreements?
- Do P2 incidents count toward the 40-review minimum in shadow mode, or only P1?

Next documents needed before coding

- **Tech Design Doc:** API contracts, database tables (`jobs` , `alerts` , `reviews`), Redis key design, Loki/Prometheus query templates, deployment and scaling.
- **Agent State Schema:** the typed LangGraph state object passed between nodes, and the exact JSON output of the Diagnosis node.

Resume point (use real numbers once measured)

Engineered an autonomous SRE agent using LangGraph to triage P1 alerts, cutting MTTR from ~30 min to under 2 min (93%). Built an async Celery/Redis pipeline, RAG over 6 months of runbooks and incidents, and an eval framework reaching 84% root-cause accuracy under 20k tokens per incident.

The 93% and 84% figures are targets in this PRD. Put them on a resume only after the eval and shadow-mode results confirm them; otherwise write "targeting".